水果分类

水果分类

- 效果介绍
- 例程源码
- 代码流程
- 代码结构
  1. 导入依赖
  2. 配置初始化
  3. 管道和模型实例化
  4. 主循环
  5. 异常处理和资源释放

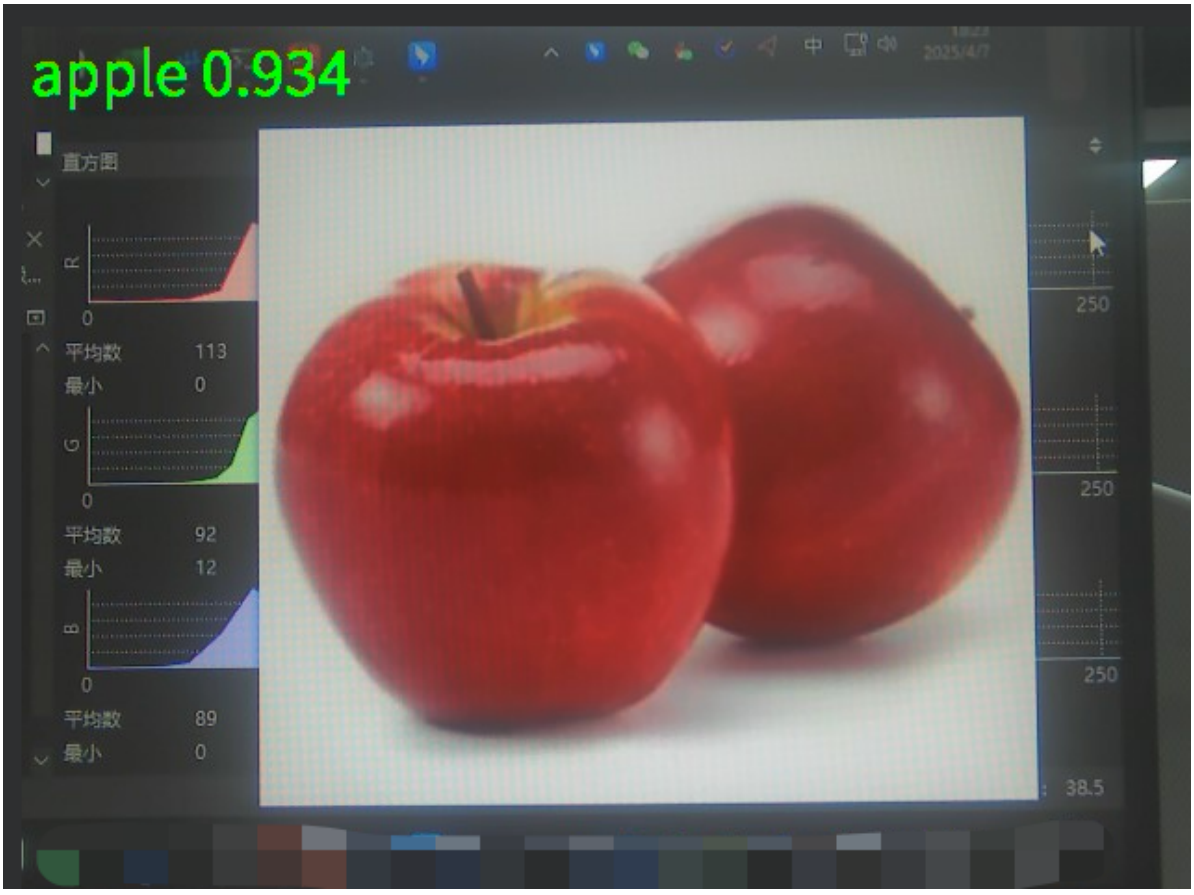
效果介绍

本节我们将使用基于yolov5训练的分类模型，来进行简单的水果分类

该模型的训练步骤可以参考【13.训练自己的模型 / 本地部署yolo训练环境】这一节教程

> 注：训练需要GPU环境，且具有一定的操作难度

识别苹果



识别香蕉



识别橘子



例程源码

```
# 导入必要的库 / Import necessary libraries
```

```
from libs.PipeLine import PipeLine, ScopedTiming  # 导入PipeLine类和计时工具 / Import PipeLine class and timing tool
from libs.YOLO import YOLOv5  # 导入YOLOv5类 / Import YOLOv5 class
import os, sys, gc  # 导入操作系统、系统和垃圾回收相关库 / Import OS, system and garbage collection libraries
import ulab.numpy as np  # 导入numpy的兼容库ulab / Import numpy compatible library ulab
import image  # 导入图像处理库 / Import image processing library

if __name__ == "__main__":  # 主程序入口 / Main program entry

    # 设置图像和显示尺寸 / Set image and display sizes
    rgb888p_size = [1280, 720]  # 原始RGB图像尺寸 / Original RGB image size
    display_size = [640, 480]  # 显示尺寸 / Display size

    # 模型路径 / Model path
    kmodel_path = "/sdcard/kmodel/fruit_cls.kmodel"  # 水果分类模型路径 / Fruit classification model path

    # 标签列表 / Label list
    labels = ["apple", "banana", "orange"]  # 可识别的水果类别 / Recognizable fruit categories

    # 置信度阈值 / Confidence threshold
    confidence_threshold = 0.5  # 分类结果置信度阈值 / Classification confidence threshold

    # 模型输入尺寸 / Model input size
    model_input_size = [224, 224]  # 模型所需的输入图像尺寸 / Required input image size for the model

    # 初始化PipeLine / Initialize PipeLine
    pl = PipeLine(rgb888p_size=rgb888p_size,
                  display_size=display_size,
                  display_mode="lcd")  # 创建图像处理流水线 / Create image processing pipeline
    pl.create()  # 创建流水线资源 / Create pipeline resources

    # 初始化YOLOv5实例 / Initialize YOLOv5 instance
    yolo = YOLOv5(task_type="classify",  # 任务类型为分类 / Task type is classification
                  mode="video",  # 视频模式 / Video mode
                  kmodel_path=kmodel_path,  # 模型路径 / Model path
                  labels=labels,  # 标签列表 / Label list
                  rgb888p_size=rgb888p_size,  # 原始图像尺寸 / Original image size
                  model_input_size=model_input_size,  # 模型输入尺寸 / Model input size
                  display_size=display_size,  # 显示尺寸 / Display size
                  conf_thresh=confidence_threshold,  # 置信度阈值 / Confidence threshold
                  debug_mode=0)  # 调试模式关闭 / Debug mode off

    # 配置预处理过程 / Configure preprocessing
    yolo.config_preprocess()

    try:
        # 主循环，持续处理视频帧 / Main loop for continuous video frame processing
        while True:
```

```
            os.exitpoint()  # 检查退出点 / Check exit point

            # 计时整个处理周期 / Time the entire processing cycle
            with ScopedTiming("total", 1):
                # 获取一帧图像 / Get a frame
                img = pl.get_frame()

                # 执行推理 / Run inference
                res = yolo.run(img)

                # 在OSD层绘制结果 / Draw results on OSD layer
                yolo.draw_result(res, pl.osd_img)

                # 显示图像 / Display image
                pl.show_image()

                # 执行垃圾回收 / Perform garbage collection
                gc.collect()

    except Exception as e:
        # 异常处理 / Exception handling
        sys.print_exception(e)  # 打印异常信息 / Print exception information

    finally:
        # 清理资源 / Clean up resources
        yolo.deinit()  # 释放YOLO资源 / Release YOLO resources
        pl.destroy()  # 销毁PipeLine资源 / Destroy PipeLine resources
```

代码流程

主要工作原理：

1. 从摄像头获取实时视频流
2. 对每一帧进行预处理，调整尺寸以适配模型输入要求
3. 将处理后的图像送入神经网络进行推理
4. 根据置信度阈值筛选分类结果
5. 在显示界面上绘制识别结果
6. 循环处理下一帧

代码结构

1. 导入依赖

```
from libs.PipeLine import PipeLine, ScopedTiming
from libs.YOLO import YOLOv5
import os,sys,gc
import ulab.numpy as np
import image
```

- `PipeLine`：处理图像流水线
- `ScopedTiming`：性能计时工具
- `YOLOv5`：封装了YOLO模型的推理接口
- `ulab.numpy`：MicroPython下的NumPy兼容库，针对嵌入式设备优化

2. 配置初始化

```
rgb888p_size=[1280,720]  # 原始图像尺寸
display_size=[640,480]   # 显示尺寸
kmodel_path="/sdcard/kmodel/fruit_cls.kmodel"  # 模型路径
labels = ["apple","banana","orange"]  # 分类标签
confidence_threshold = 0.5  # 置信度阈值
model_input_size=[224,224]  # 模型输入尺寸
```

3. 管道和模型实例化

```
pl=PipeLine(rgb888p_size=rgb888p_size,display_size=display_size,display_mode="lc
d")
pl.create()
yolo=YOLOv5(task_type="classify",mode="video",kmodel_path=kmodel_path,labels=lab
els,
            rgb888p_size=rgb888p_size,model_input_size=model_input_size,

display_size=display_size,conf_thresh=confidence_threshold,debug_mode=0)
yolo.config_preprocess()
```

4. 主循环

```
try:
    while True:
        os.exitpoint()
        with ScopedTiming("total",1):
            img=pl.get_frame()       # 获取一帧
            res=yolo.run(img)        # 运行推理
            yolo.draw_result(res,pl.osd_img)  # 绘制结果
            pl.show_image()          # 显示图像
            gc.collect()             # 垃圾回收
```

5. 异常处理和资源释放

```
except Exception as e:
    sys.print_exception(e)
finally:
    yolo.deinit()
    pl.destroy()
```